SW Team - Doxygen Guidelines

Page Content

- General
  - What to Document
    - Mandatory
    - Optional
  - How to Document C
  - How to Document CAmkES
  - Templates
- Rules
  - General
    - doxy_general_1
    - doxy_general_2
    - doxy_general_3
    - doxy_general_4
    - doxy_general_5
  - Docu
    - doxy_docu_1
    - doxy_docu_2
    - doxy_docu_3
    - doxy_docu_4
    - doxy_docu_5
    - doxy_docu_6
    - doxy_docu_8
  - Grouping
    - doxy_grouping_1
    - doxy_grouping_2
- Project Initialization
  - Installation
  - Setup
- References

General

The purpose of this guideline is to create properly documented code with a consistent appearance.

Please always keep in mind that main audience are SDK users, and documentation should contain information useful for them.

What to Document

Doxygen, by default, will only parse files that are marked with the `@file` attribute (unless EXTRACT_ALL is enabled) thus any file with this attribute shall contain doxygen style documentation.

Mandatory

- Public features of a module shall be documented, i.e. mainly header files. (If something does not require documentation, it should be moved to the source file or internal header file.)
- CAmkES files for components and interfaces shall be documented.

Optional

- Doxygen may be used to to document "private" features in a source file as well but don't expect it to be included in the generated user documentation.
- General documentation may be put in markdown files (i.e. files with *.md extension) which will also be parsed by doxygen and result in separate pages in the doxygen documentation.

How to Document C

For different entities in C the documentation shall answer the following questions:

- Files / Groups
  - What is their responsibility?
  - How do they fit into the architecture?
- Functions / Macros
  - What do they do?
  - Who is supposed to use them (private, public)?
  - Input parameters (type, direction and purpose).
  - Return values (type, error case).

- Variables / Defines
    - Who uses them?
    - What do they control?
- Structures
    - Who uses them?
    - Are they public or private?

How to Document CAmkES

Although not strictly enforced by CAmkES, three different types of files can be distinguished:

1. Components
2. Interfaces
3. Assemblies

Components and interfaces files shall include documentation of files, components/interfaces (optional if it duplicates the file description) and members.

The documentation of assembly files is optional. If present it should describe the purpose of the application at file level but must not include documentation for internal connections and configurations.

Templates

The following repository contains templates of files that can be documented with Doxygen:

> https://bitbucket.hensoldt-cyber.systems/projects/HC/repos/doc_templates/browse

The templates are meant to reflect what is described in this guideline and can be considered as its cheat-sheet.

NOTE: If anything is contradicting, the guideline has a higher priority and templates should be updated.

Rules

Doxygen syntax is documented here:

> http://www.doxygen.nl/manual/docblocks.html

For the documentation, to give a good impression, the following rules must be followed so that consistency is kept.

General

doxy_general_1

The following doxygen style shall be used:

1. Javadoc style comments (`/**`) for comments with doxygen commands and comments for visual separation.
2. C++ style comments (`//!`) for compact comments above an entity.
3. C++ style inline-comments (`//!<`) for compact comments behind the entity.

```
/**
 * Abstracts Foo.
 */
typedef struct
{
    int x; //!< X coordinate.
    int y; //!< Y coordinate.
    int z; //!< Z coordinate.
} Foo_t;

//! Number of elements in the foo pool.
extern const int fooPoolLen;
```

Rational

Chosen Style shall be consistent across all our projects for aesthetic purposes.

- Javadoc style is widely used and very much C-like and also gives a nice visual separation of the entities.
- C++ style comment ('//!') is more compact compared to Javadoc style.

doxy_general_2

The `@` doxygen command prefix shall be used.

```
/**
 * @see https://www.doxygen.nl/manual/commands.html#cmdsee
 */
```

Rational

Either `\` or `@` has to be selected, and `@` looks more verbose.

doxy_general_3

The following doxygen commands shall not be used:

Unused Commands	Rational
`@addtogroup`	If used with the same label it will silently merge the documentation and not result in a warning, which is not desired.
`@author`, `@authors`	Git is used for tracking this.
`@copyright`	Separate copyright header used instead, see doxy_general_4.
`@date`	Git is used for tracking this.
`@returns`	@return is used instead, see doxy_docu_5.
`@version`	Git is used for tracking this.

doxy_general_4

A separate copyright header shall be added above the doxygen header.

Copyright headers shall not be a doxygen style comment and not use `@copyright` command.

If there is no doxygen header in the file, the copyright header may include a short description, e.g. the module name. For longer descriptions a doxygen header should be added.

The year in the copyright header shall be the **year the file was created**.

- If substantial changes have been done in several years, a **year range <first year>-<last year>** should be used.
- If in doubt whether a change is substantial, prefer the year range.

```
/*
 * Copyright (C) 2020-2021, HENSOLDT Cyber GmbH
 */

/*
 * Module XYZ
 *
 * Copyright (C) 2020-2021, HENSOLDT Cyber GmbH
 */
```

Rational

Adding copyright information in the doxygen comments places it in the generated documentation what does not look good and is not desired.

doxy_general_5

CAmkES files shall also be documented with doxygen.

Therefore the custom extensions shall be configured in the Doxyfile to enable documenting those files.

```
# add custom extension
FILE_PATTERNS          += *.camkes

# add the custom extension to C code doxygen parser
EXTENSION_MAPPING      += camkes=C
```

Rational

Since CAmkES syntax is very much similar to the C/C++ syntax it would be a pity not benefit from the doxygen.

Docu

doxy_docu_1

A `@file` command shall be added at the top of the file to mark files for generating documentation.

In most cases header files and CAmkES files shall be marked with `@file` command.

```
/**
 * @file
 *
 * Describes the file briefly.
 *
 * Describes the file in detail.
 */
```

Rational

`@file` is required to select parts of the projects to be documented.

doxy_docu_2

Brief description shall always be present, in the present active voice, and terminated with a dot.

Since auto-brief is enabled the `@brief` command should not be used.

```
/**
 * Initializes the system.
 */
void initSystem(void);
```

Rational

Brief descriptions are needed when entities are listed in the doxygen documentation - the second column of the list is the brief description.

Auto-brief feature is helpful when using inline comments but requires a description to be terminated with a dot.

doxy_docu_3

Groups of doxygen commands shall be space aligned.

```
/**
 * Calls `convert` function implementation.
 *
 * @retval OS_ERROR_INVALID_PARAMETER `self` or `entry` is a NULL pointer.
 * @retval OS_SUCCESS                 Operation was successful.
 * @retval other                      Implementation specific.
 *
 * @param[out] self  Result of the conversion will be stored in the underlying
 *                   buffer.
 * @param[int] entry Log entry to be converted to the given format.
*/
OS_Error_t
FormatT_convert(
    OS_LoggerAbstractFormat_Handle_t* self,
    OS_LoggerEntry_t const* const     entry
);
```

Rational

Alignment improves readability and space alignment is more compact compared to tab alignment.

doxy_docu_4

Structures, enums, etc. shall be commented with Javadoc comments `/**`.

Function arguments, structure members, enum values, variables, etc. shall be commented with C++ style inline comments after the entity (`//!<`).

If an inline comment does not fit into one line, function arguments shall be documented with `@param` or C++ style comments (`//!`) may be used directly above structure members, enum values, etc.

```
/**
 * Abstracts Foo
 */
typedef struct
{
    int x; //!< X coordinate.
    int y; //!< Y coordinate.
    int z; //!< Z coordinate.
} Foo_t;

/**
 * Rotates given Foo.
 */
void rotate(
    Foo_t*,     //!< [in] Pointer to the object to be rotated.
    int degrees //!< [in] Angle of rotation in degrees.
);

/**
 * Copies Lorem ipsums.
 *
 * @param[in] destination Pointer to the destination object.
 * @param[in] source      Pointer to the source object.
 */
void copy(
    LoremIpsumDolorSitAmetConsecteturAdipiscingElitPraesentVariu_t* destination,
    LoremIpsumDolorSitAmetConsecteturAdipiscingElitPraesentVariu_t* source
);

/**
 * Calls `convert` function implementation.
 *
 * @param[out] self  Result of the conversion will be stored in the underlying
 *                   buffer.
 * @param[int] entry Log entry to be converted to the given format.
 *
 * @retval OS_ERROR_INVALID_PARAMETER `self` or `entry` is a NULL pointer.
 * @retval OS_SUCCESS                 Operation was successful.
 * @retval other                      Implementation specific.
*/
OS_Error_t
FormatT_convert(
    OS_LoggerAbstractFormat_Handle_t* self,
    OS_LoggerEntry_t const* const     entry
);

Foo_t foo; //!< Global foo.

//! This is a description of a variable that does not fit in a inline comment,
//! it even needs two lines.
BarWithAPrettyLongTypeName_t barHasALongVariableNameAsWell;
```

Rational

Javadoc comments give a nice visual separation for structs and enums.

Inline comments for function arguments avoid the error-prone repetition of argument names.

C++ comments, especially inline comments are more compact.

doxy_docu_5

The direction of function arguments shall be specified with `[in]`, `[out]` or `[in,out]`.

```
/**
 * Copies count bytes from source to destination.
 *
 * @return ...
 */
void*
memcpy(
    void* dest,      //!< [out]    Pointer to the memory location to copy to.
    const void* src, //!< [in]     Pointer to the memory location to copy from.
    size_t count,    //!< [in]     Number of bytes to copy.
    int* magic       //!< [in,out] Argument to demonstrate in,out parameters.
);
```

Rational

C language does not have a strong notation of which arguments are input/outputs, so it is a good practice to document this explicitly.

doxy_docu_6

The return values of a function shall be document either with `@return` or with `@retval`.

- `@return` shall be used to explain what the return value means (e.g. handle, value, ...).
- `@retval` shall be used to list distinct return values (e.g. error codes, states, ...).
- `@return` and `@retval` shall be placed BEFORE function parameters documentation.

NOTE: If a function returns `OS_Error_t` all expected error codes shall be listed with `@retval`. If it is not possible to list all error codes, one `@retval` shall state that there are other possible error codes (e.g. for implementation specifics).

```
/**
 * Gets Foo.
 *
 * @return Pointer to valid Foo that will never be NULL.
 */
Foo_t*
getFoo(void);

/**
 * Checks if Foo.
 *
 * @retval true  Given object is Foo.
 * @retval false Given object is not Foo.
 */
bool
isFoo(
    void* foo //!< The given Foo.
);

/**
 * Calls `convert` function implementation.
 *
 * @retval OS_ERROR_INVALID_PARAMETER Self or entry is a NULL pointer.
 * @retval OS_SUCCESS                 Operation was successful.
 * @retval other                      Implementation specific.
 *
 * @param[out] self  Result of the conversion will be stored in the underlying
 *                   buffer.
 * @param[in]  entry Log entry to be converted to the given format.
 */
OS_Error_t
FormatT_convert(
    OS_LoggerAbstractFormat_Handle_t* self,
    OS_LoggerEntry_t const* const     entry
);
```

Rational

It is very important to clearly document what the function returns and what values can be expected.

doxy_docu_8

Parts of the code that cannot be parsed by doxygen shall be excluded with the `@cond` and `@endcond` or `@hideinitializer` commands.

`@cond` and `@endcond` shall be used to skip CAmkES imports, as otherwise doxygen will warn undocumented entities.

```
/** @cond SKIP_IMPORTS */
import "Interface.camkes";
/** @endcond */
```

`@hideinitializer` shall be used to avoid extracting component members as its initialization values.

```
/**
 * ...
 *
 * @hideinitializer
 */
component Component {
    ...
}
```

Rational

There are cases when we need to exclude certain parts from being documented because doxygen cannot handle it.

Grouping

doxy_grouping_1

Files shall be grouped in the Doxygen documentation.

Grouping is documented here:

> http://www.doxygen.nl/manual/grouping.html

Rational

Grouping helps having a view of the system architecture and makes the documentation cleaner.

Examples

- `@defgroup` shall be used to define main group:

  ```
  // main.h
  /**
   * @file
   *
   * ...
   *
   * @ingroup  main
   * @defgroup main Main module
   *
   * Group's brief description.
   *
   * Group's detailed description.
   */
  ```

- `@ingroup` shall be used to add other files to the `main` group:

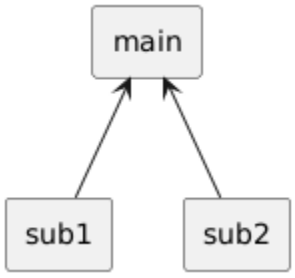
```
// foo.h
/**
 * @file
 *
 * ...
 *
 * @ingroup main
 */
```

- `@defgroup` shall be used to define sub-groups and `@ingroup <MAIN_GROUP_NAME>` to nest them below the main module:

```
// sub1.h
/**
 * @file
 *
 * ...
 *
 * @ingroup  sub1
 * @defgroup sub1 Sub module 1
 *
 * Group's brief description.
 *
 * Group's detailed description.
 *
 * @ingroup main
 */

// sub2.h
/**
 * @file
 *
 * ...
 *
 * @ingroup  sub2
 * @defgroup sub2 Sub module 2
 *
 * Group's brief description.
 *
 * Group's detailed description.
 *
 * @ingroup main
 */
```

- The result shall be a structure that reflects the software architecture:



doxy_grouping_2

Member grouping with `@{` and `@}` shall be used to reuse one documentation for similar entities.

```
/**
 * Asserts equality of the expected and actual values.
 *
 * This macro assert equality expected and actual values of a given type.
 * @{
 */
#define ASSERT_EQ_UINT(expected, actual) ASSERT_EQ(expected, actual, "u")
#define ASSERT_EQ_INT (expected, actual) ASSERT_EQ(expected, actual, "d")
#define ASSERT_EQ_SZ  (expected, actual) ASSERT_EQ(expected, actual, "zu")
/** @} */
```

Rational

Helps to follow the Don't Repeat Yourself principle, and improves source file readability.

Project Initialization

Installation

Doxygen should be part of the docker container used as build environment.

Alternatively, doxygen can be installed locally (optionally Graphviz and the doxygen wizard):

```
sudo apt-get install doxygen graphviz doxygen-gui
```

Setup

- Doxygen configuration can be fine tuned with a help of the Doxygen wizard or with the template Doxyfile as a starting point:
  - Project
    - project name
    - source code directory
    - destination directory
    - and set the option to scan directories recursively
  - Mode
    - set the option to optimize for C output
  - Diagrams
    - set the option to use dot tool from GraphViz package (make sense, if you want to have a view of dependent objects)
- Run the doxygen tool and check the output.

References

- https://bitbucket.hensoldt-cyber.systems/projects/HC/repos/doc_templates
- http://www.doxygen.nl/manual/
- http://www.doxygen.nl/manual/docblocks.html
- http://www.doxygen.nl/manual/grouping.html
- SW Team - 3rd Party Module Guidelines#CopyrightHeader